POSTASSIST LK · SMART REGISTERED MAIL INTAKE AND TRACKING

The IoT Scale and Camera

Everything about the IoT part of the component, in simple words: the parts, the wiring, how to program it, how the program works line by line, what the backend does with it, how it is tested, and what to say to the panel.

WHAT IS INSIDE

1 The IoT part in one minute

2 The parts and what each one does

3 Wiring

4 How weighing works

5 How to program it, step by step

6 How the program is organised

7 The program, piece by piece

8 The device's own web page and serial commands

9 What the backend does with it

10 Calibration and accuracy testing

11 Using it at the counter

12 Security and reliability

13 Troubleshooting

14 Your device right now – to-do list

15 Limitations

16–17 Panel questions · IoT words

HOW TO READ THIS GUIDE

For the panel, Part 1 and Part 16 are enough. To understand the programming, read Parts 5–8: the code shown there is the real firmware, shortened and with plain-English comments. Every number in this guide (thresholds, timings, pins) is taken from the actual code in `iot/postassist_scale_cam/` and `backend/`.

Component owner
Senarathna W.L.S.K. · IT23324138
B.Sc. (Hons) in Information Technology, SLIIT

Firmware
pa-scalecam-1.1.1 (repository version)
English · October 2026

PART 1

The IoT part in one minute

What it is, why it exists, and the short answer for the panel.

The IoT part is **one small device that is both a scale and a camera**. A registered letter is placed on its platform. The device:

1. **weighs** the letter with a load cell and an HX711 amplifier,
2. **sends** the weight to the backend over Wi-Fi **twice every second**, and
3. **photographs** the envelope from above when the counter officer asks for it, so the system can read the address.

The counter screen shows the live weight. The officer presses **Use this weight**, and the backend accepts it only when the reading is fresh, steady and calibrated, and has not been used for another letter. The weight then goes straight into the price calculation – nobody reads a scale or types a number.

WHY ONE DEVICE FOR BOTH JOBS

The weight and the photo of the **same** envelope come from the **same** device at the **same** place – the envelope is photographed while it lies on the scale. That keeps the cost low and makes it impossible to mix the photo of one letter with the weight of another.

THE SHORT ANSWER FOR THE PANEL

"The IoT part is a smart scale with a camera, built from an ESP32-CAM board, a 1 kg load cell and an HX711 amplifier. The firmware, which I wrote in Arduino C++, reads the scale ten times a second, turns the reading into grams using the tare offset and the calibration factor, and decides whether the weight is stable. Every half second it sends the reading to my FastAPI backend as JSON, with its own device key. The server's reply carries any commands – tare, calibrate or take a photo – which the device carries out and reports back. At the counter the weight can be used only if it is fresh, stable, calibrated and not already used for another letter."

PART 2

The parts and what each one does

What you bought, what else is needed, and why.

2.1 The main parts (purchased, Rs 4,800 in total)

Part	What it does	Price
ESP32-CAM (AI-Thinker) with OV3660 3 MP camera	The **brain**. A tiny computer on one chip with Wi-Fi, 4 MB of extra memory (PSRAM) for photos, and a camera. It runs your program (the *firmware*), reads the scale, takes photos and talks to the backend.	Rs 2,750
1 kg bar load cell	A metal bar with *strain gauges* glued on it. When weight pushes on one end, the bar bends a tiny amount and its electrical resistance changes. Its maximum is 1 kg; the system uses 5–500 g.	Rs 450
HX711 module	A 24-bit *analogue-to-digital converter* (ADC) with a built-in amplifier (gain 128). The load cell gives only a few millivolts; the HX711 turns that into a number about **10 times per second**.	Rs 160
Wooden bracket	Base and platform. One end of the load cell is screwed to the base, the other end carries the platform, so the cell can bend.	Rs 600
4-channel level shifter	Translates signals between **5 V** (HX711 side) and **3.3 V** (ESP32 side). ESP32 pins are damaged by 5 V, so the data and clock wires pass through it.	Rs 100
Jumper wires + shipping	Connections between the modules.	Rs 740

2.2 Extra items needed

Item	Why
ESP32-CAM-MB adapter board (micro-USB, IO0 button)	The ESP32-CAM has no USB socket. This board plugs underneath it so you can **upload the program** from the laptop and see its messages.
Breadboard, male–female jumpers	Shared 5 V and GND lines for all modules.
Micro-USB breakout + 5 V / 2 A phone charger	Power. Wi-Fi and the camera draw current spikes; a weak supply makes the board reset (“brownout”).
1000 µF capacitor	Placed across 5 V and GND next to the ESP32 to absorb those current spikes.
Reference masses (e.g. 10–500 g set)	Needed to **calibrate** and to **test accuracy**. Coins and kitchen scales are not reliable references.
Camera stand + desk lamp	Holds the camera 28–32 cm above the platform, lens down; the lamp gives even light without glare.
Data USB cable	A charge-only cable cannot upload the program.

PART 3

Wiring

How the modules are connected (Option A – HX711 at 5 V through the level shifter).

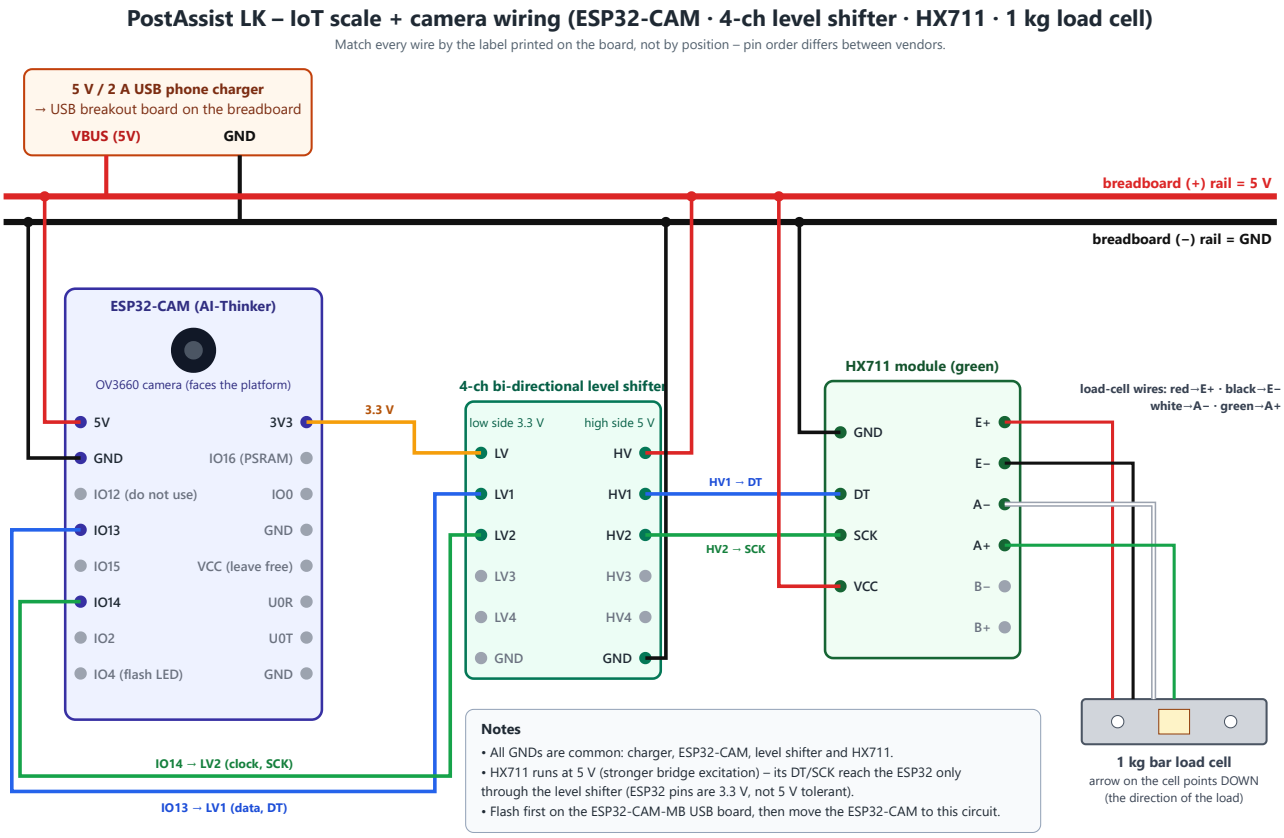


Figure 1 – The complete wiring (also in Documents/IoT_Hardware_Setup_Guide.md, Step 6).

From	To	Why
Charger 5 V / GND (via USB breakout)	breadboard + rail / − rail	Power for everything
ESP32-CAM **5V** and **GND**	+ rail and − rail	Powers the ESP32-CAM
ESP32-CAM **3V3**	level shifter **LV**	3.3 V reference for the low side
level shifter **HV** / **GND**	+ rail / − rail	5 V reference for the high side
HX711 **VCC** / **GND**	+ rail / − rail	HX711 at 5 V gives the load cell a stronger signal
HX711 **DT** → shifter **HV1**	shifter **LV1** → ESP32 **IO13**	Data from the HX711
HX711 **SCK** → shifter **HV2**	shifter **LV2** → ESP32 **IO14**	Clock pulses from the ESP32
Load cell red / black / white / green	HX711 E+ / E− / A− / A+	Excitation (power) and signal of the load cell
1000 µF capacitor (+ leg to 5 V)	across + and − rails	Stops brownout resets

WHY EXACTLY THESE PINS

- **GPIO 13 and 14** are free because the program does not use the SD-card slot – almost every other pin is taken by the camera and PSRAM.
- **Never use GPIO 12** (it decides how the chip boots) or **GPIO 16** (used by PSRAM). **GPIO 4** is the white flash LED and **GPIO 33** the small red status LED.
- Never feed 5 V into the ESP32's **3V3** or **VCC** pins, and never connect DT/SCK directly at 5 V.
- **Option B** (if the shifter fails): power the HX711 from 3.3 V and connect DT/SCK directly to IO13/IO14 – safe, but a little noisier.

PART 4

How weighing works

From a letter on the platform to the number on the counter screen.

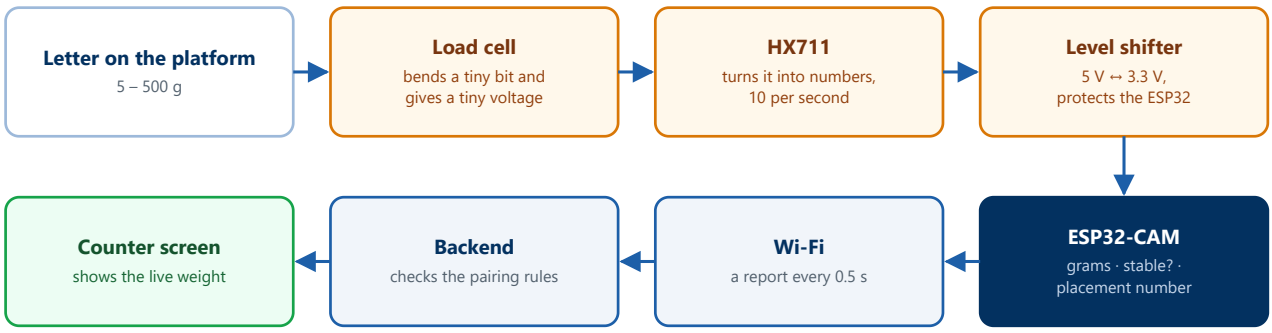


Figure 2 – The weighing chain.

1. The letter pushes the platform down; the **load cell bends** by a fraction of a millimetre.
2. The strain gauges change resistance, so the cell's output changes by a few **millivolts**.
3. The **HX711** amplifies it (×128) and converts it to a **24-bit number** (a "raw count"), about 10 times a second.
4. The **ESP32** reads that number bit by bit through the level shifter.
5. The program turns raw counts into **grams** with two saved values: the **offset** (the raw count of the empty platform, set by *tare*) and the **factor** (raw counts per gram, set by *calibrate*).
6. It decides whether the weight is **stable** and **loaded**, numbers the **placement**, and sends everything to the backend every 0.5 s.

WORKED EXAMPLE – HOW GRAMS ARE CALCULATED

1. Tare with the platform empty: average raw count	84,300	→ offset = 84,300
2. Calibrate with a 100 g mass: average raw count	299,300	→ factor = (299,300 − 84,300) ÷ 100 = **2,150 counts per gram**
3. A letter on the platform: average raw count	127,300	→ weight = (127,300 − 84,300) ÷ 2,150 = **20.0 g**

grams = (raw count − offset) ÷ factor. The numbers above are an illustration; your real offset and factor appear on the device page (`/status`).

4.1 The words “stable”, “loaded” and “placement”

Word	Rule in the program
Window	The last **10 readings** = 1 second.
Spread	Highest minus lowest reading in the window, in grams – how much it wobbled.
Stable	The window is full **and** the spread is **1 g or less**.
Loaded	Weight is **2 g or more**.
Empty	Stable and below **1 g**.
Placement number	Goes up by 1 when a stable, loaded reading appears after the platform was empty. Saved in flash, so it keeps increasing across restarts.

PART 5

How to program it, step by step

Putting the firmware on the ESP32-CAM.

5.1 One-time set-up on the laptop

1. Install **Arduino IDE 2**.
2. *File → Preferences → Additional boards manager URLs*: add `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
3. *Tools → Board → Boards Manager*: install **esp32 by Espressif Systems**, version **3.x** (the firmware was built and tested with 3.3.12; your IDE had 2.0.17 – update it).
4. *Tools → Manage Libraries*: install **ArduinoJson** by Benoit Blanchon, version **7**. No HX711 library is needed – the program reads the chip itself.
5. Keep the Arduino *sketchbook* folder on a path without special characters (your old one under `OneDrive\ドキュメント` made the libraries invisible).

5.2 The files

File (in iot/postassist_scale_cam/)	What it is
`postassist_scale_cam.ino`	The main program (about 700 lines).
`config.h`	**Your settings** – Wi-Fi, backend address, device ID and key, pins, thresholds. Made by copying `config.example.h`. It is **not** stored in git because it holds secrets.
`config.example.h`	The template for `config.h` (should contain only placeholders).
`device_page.h`	The HTML and JavaScript of the device's own web page.
`camera_pins.h`	Which ESP32 pins the camera connector uses (AI-Thinker layout).

5.3 Settings in config.h

Setting	What to put
`WIFI_SSID` / `WIFI_PASSWORD`	A **2.4 GHz** network (the ESP32 cannot join 5 GHz). Your iPhone hotspot works.
`API_BASE_URL`	The laptop's address on that network, e.g. `http://172.20.10.5:8000`. Not `localhost` – for the ESP32 that means itself.
`DEVICE_ID`	`SCALECAM-01` (same as `SEED_DEVICE_ID` in `backend/.env`).
`DEVICE_KEY`	Same as `SEED_DEVICE_KEY` in `backend/.env`.
`ROOT_CA_PEM`	Only for an `https://` server; leave empty in the lab.
Pins and thresholds	Leave the defaults (DT 13, SCK 14, window 10, stable 1 g, report every 500 ms …).

5.4 Upload

1. Seat the camera ribbon firmly. Plug the ESP32-CAM onto the **MB** board and connect it to the laptop with a **data** cable.
2. Open `postassist_scale_cam.ino`. In *Tools* choose: Board **AI Thinker ESP32-CAM** · Partition Scheme **Huge APP (3MB No OTA/1MB SPIFFS)** · PSRAM **Enabled** · Upload speed 460800 (115200 if it fails) · the **COM port** of the MB board.
3. Press **Upload** (the arrow). If it says *Failed to connect to ESP32*: hold **IO0** on the MB board, press and release **RST**, release IO0, upload again.
4. After "Done uploading", press **RST**. Open *Serial Monitor* at **115200 baud**.

You should see something like this (it prints the IP address the hotspot gave the board):

```
PostAssist scale+camera pa-scalecam-1.1.1 (SCALECAM-01)
[cam] sensor PID 0x3660
[wifi] connecting to Senomie's iPhone
[wifi] connected, IP 172.20.10.2
[scale] calibration CAL-A235-92298047, factor 2150.000, offset 84300
[api] http://172.20.10.5:8000
Type 'i' for info, 't' to tare, 'c 100' to calibrate with a 100 g mass
```

5.5 After uploading

1. Take the ESP32-CAM off the MB board and put it back in the circuit (the MB board is only for programming).
2. Power the circuit. Open `http://<device-IP>/` – you should see the live weight and `sensor_ok: true`. Press the platform: the number must go **up** (if it goes down, swap the white and green load-cell wires).
3. Start the backend (`--host 0.0.0.0`) and the web app. The **Scale & camera** page (sidebar: Set-up) should show **online** within a second.
4. **Tare** with the platform empty, then **Calibrate** with a 100 g mass (Part 10). Done once – it is saved in flash.

> **CHANGING WI-FI LATER NEEDS NO RE-FLASHING**
>
> The device page has a **Network** box: enter a new Wi-Fi name, password or backend address and the board saves them and restarts. If it cannot join any Wi-Fi within 20 seconds, it opens its own network **PostAssist-Scale** (password in `SETUP_AP_PASSWORD`, default `postassist`): join it with a phone and open `http://192.168.4.1/`.

PART 6

How the program is organised

Every Arduino program has two parts: `setup()` and `loop()`.

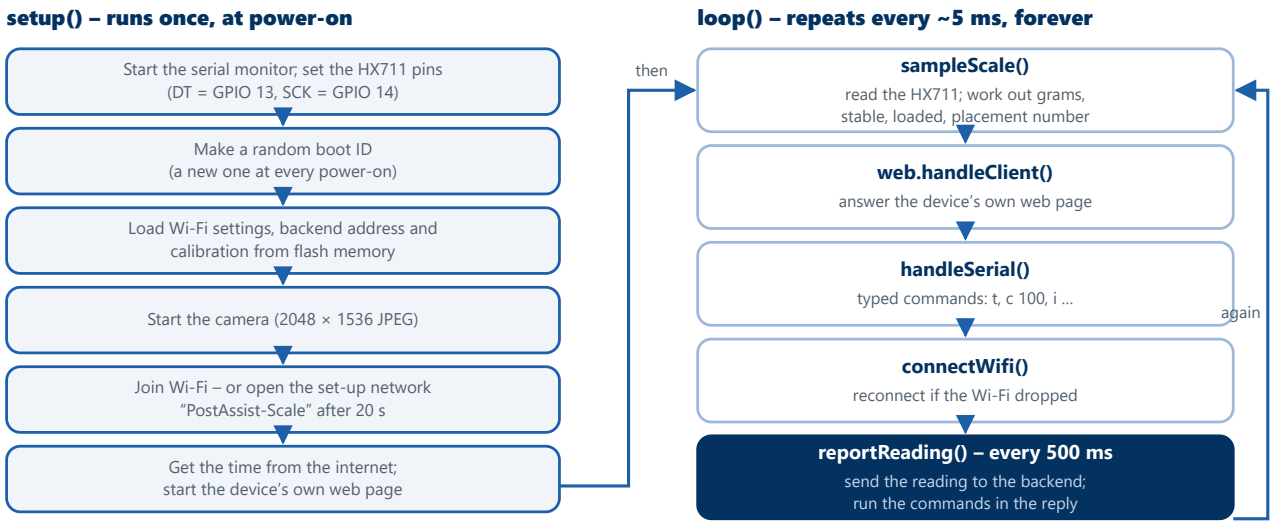


Figure 3 – `setup()` prepares everything once; `loop()` then runs about 200 times per second.

The real `loop()` is only a few lines:

```
void loop() {
  sampleScale();                // read the HX711 and update grams / stable / loaded
  web.handleClient();           // answer the device's own web page
  handleSerial();               // typed commands from the serial monitor
  connectWifi(false);           // reconnect if the Wi-Fi dropped (tries every 15 s)
  if (WiFi.status() == WL_CONNECTED && millis() - lastReportMs >= REPORT_INTERVAL_MS) {
    lastReportMs = millis();
    reportReading();            // every 500 ms: send to the backend, run the commands in the
reply
  }
  delay(5);
}
```

6.1 The libraries it uses

Library	Used for
`WiFi`	Joining the Wi-Fi, or opening the set-up network.
`HTTPClient` (+ `WiFiClientSecure`)	Sending readings, photos and command results to the backend (HTTP, or HTTPS with a root certificate).
`ArduinoJson`	Building the JSON of a reading and reading the JSON reply with commands.
`Preferences`	Saving the calibration, placement number and Wi-Fi settings in **flash memory (NVS)**, so they survive power cuts.
`WebServer`	The device's own web page on port 80.
`esp_camera`	Starting the camera and taking JPEG photos.

6.2 What is kept in flash memory

Group (“namespace”)	Values
`scale`	`offset` (zero point), `factor` (counts per gram), `cal_id` (calibration ID), `placement` (last placement number)
`net`	`ssid` , `pass` , `api` – only if they were changed on the device page; otherwise the values from `config.h` are used

PART 7

The program, piece by piece

The real code, shortened, with plain-English comments.

7.1 Reading the HX711 – `hxRead()`

The HX711 has no “send me the number” command. The ESP32 pulls the number out **one bit at a time**: it raises the clock pin (SCK), reads the data pin (DT), lowers the clock, and repeats. This is called *bit-banging*.

```
long hxRead() {
  uint32_t value = 0;
  for (int i = 0; i < 25; i++) {              // 24 data bits + 1 extra pulse
    digitalWrite(SCK, HIGH);                  // clock up
    delayMicroseconds(4);                     // wait – slow on purpose
    if (i < 24) value = (value << 1) | digitalRead(DT);   // add one bit
    digitalWrite(SCK, LOW);                   // clock down
    delayMicroseconds(4);
  }
  if (value & 0x800000) value |= 0xFF000000;   // a negative 24-bit number stays negative
  return (long)value;
}
```

- The **25th pulse** tells the HX711 to use input A with gain 128 for the next reading.
- The program reads only when the HX711 is ready (DT goes LOW), about 10 times a second.
- **Why slow pulses (4 µs)?** Through the level shifter and jumper wires the signal edges are slow; fast reading picked up wrong bits (readings jumping by fixed amounts). 4 µs is still far below the ~50 µs after which a HIGH clock would switch the HX711 off.
- The bits are read inside a “critical section”, so Wi-Fi interrupts cannot break the timing.
- This is why no HX711 library is used: the library’s fast timing caused the bit errors.

7.2 From raw counts to grams – `sampleScale()`

```
// called on every loop
if (hxReady()) {
  raw = hxRead();
  if (raw jumps more than 25 g from the last accepted value)
      wait: accept it only if the NEXT reading agrees      // ignores single electrical spikes
  else
      add raw to the window (last 10 readings)
} else if (no reading for 1 second) {
  sensorOk = false;                                        // HX711 wiring or power problem
}

average = mean of the window without its highest and lowest value    // "trimmed mean"
netG    = (average - cal.offset) / cal.factor;               // GRAMS
spreadG = (highest - lowest) / cal.factor;                   // wobble in grams
stable  = (window has 10 readings) && spreadG <= 1.0;
loaded  = netG >= 2.0;

if (stable && loaded && wasEmpty) { placementId++; wasEmpty = false; savePlacement(); }
if (stable && netG < 1.0) {
  wasEmpty = true;
  if (|netG| < 0.5) slowly move cal.offset towards average;  // "zero tracking" of slow drift
}
```

- **Trimmed mean**: dropping the highest and lowest reading removes one-off noise.
- **Spike rejection**: a real letter is confirmed by the very next reading; an electrical glitch is not. If readings keep jumping (5 in a row), the window is cleared so the scale reports “not stable” instead of an old value.
- **Zero tracking**: while empty and within 0.5 g of zero, the offset follows slow drift (temperature), 1/20 of the difference at a time.

7.3 Tare and calibrate – `tare()`, `calibrate()`

```
// TARE - platform empty and stable: remember this raw value as zero
cal.offset = average;
saveCalibration();                                           // to flash

// CALIBRATE - known mass on the platform and stable
factor = (average - cal.offset) / knownGrams;                // raw counts per gram
if (|factor| < 100) refuse;                                  // "no usable change detected"
cal.factor = factor;
cal.id = "CAL-" + random + "-" + time;                       // a new calibration ID
saveCalibration();
```

- Tare and calibrate both need a **stable** reading. Calibrate accepts 5–1000 g on the device (the web app asks for 10–1000 g).
- The **calibration ID** is sent with every reading and stored with every paired weight, so you can always tell which calibration produced a letter’s weight.
- **Safety added in 1.1.1**: at start-up a stored factor below 100 counts/g is thrown away (back to the default 2000). Too small a factor makes normal noise look like ±25 g, the scale never becomes stable, and

tare/calibrate (which need stable) could not fix it. There is now **Reset calibration** (serial `r`, `POST /calibrate/reset`) and **force tare** (serial `tf`, `POST /tare?force=1`).

7.4 Sending a reading – `reportReading()`

Every 500 ms the program builds a JSON message and sends it with an **HTTP POST** to `/api/iot/readings`. Two headers prove who it is: `X-Device-Id: SCALECAM-01` and `X-Device-Key: <secret>`.

```
{"seq": 812, "boot_id": "3f9a01c2", "uptime_ms": 405233,
 "captured_at": "2026-10-19T04:15:02Z",
 "net_g": 20.0, "raw": 127300, "spread_g": 0.38,
 "stable": true, "loaded": true, "placement_id": 57,
 "calibration_id": "CAL-A235-92298047", "sensor_ok": true, "sample_count": 10,
 "rssi": -58, "fw": "pa-scalecam-1.1.1", "ip": "172.20.10.2", "camera_ok": true}
```

Field	Meaning
`seq` · `boot_id`	Message number, and a random ID chosen at every power-on.
`net_g` · `spread_g` · `raw`	Grams, wobble in grams, and the raw count.
`stable` · `loaded` · `placement_id`	The rules of Part 4.1.
`calibration_id` · `sensor_ok`	Which calibration is in use (empty = not calibrated); whether the HX711 answers.
`rssi` · `fw` · `ip` · `camera_ok`	Wi-Fi signal strength, firmware version, address, camera status – shown on the Scale & camera page.

The backend replies, for example: `{"server_time": "…", "commands": [{"id": "CMD-…", "type": "tare", "params": {}}]}`. The small red LED is on while the backend accepts readings.

7.5 Carrying out commands – `handleCommands()`

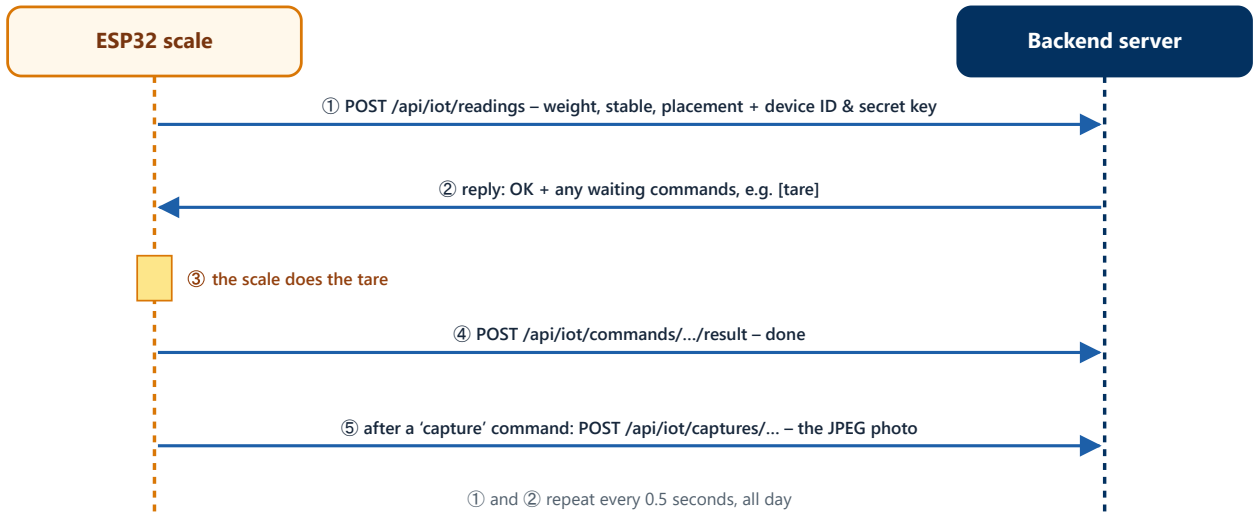


Figure 4 – Commands ride back on the reply to a reading, so nothing has to connect *to* the device.

Command	What the device does	Reports back
tare	Sets the zero point (needs stable).	message, offset
calibrate	Uses `known_g` from the command to compute a new factor.	message, calibration_id, factor
capture	Takes the best photo and uploads it (7.6).	bytes, width, height, HTTP code
identify	Blinks the red LED 3 times (to find the board).	–
reboot	Reports first, then restarts.	–

Each result is sent to `POST /api/iot/commands/<id>/result` as `{"status": "done" | "failed", "result": {…}}`.

7.6 The camera – `initCamera()`, `captureBest()`, `uploadCapture()`

- **Start-up settings**: 2048 × 1536 pixels (QXGA) JPEG at quality 10, two frame buffers in PSRAM, automatic white balance and exposure, gain limited to 4× (less grain), bad-pixel and lens-shading correction, **sharpness +2**, **noise reduction off** (it would blur thin pen strokes), image flipped because the module hangs upside down. Without PSRAM it falls back to 800 × 600.
- **Taking a photo**: throw away 3 warm-up frames (exposure settles), take a burst of 3, and keep the **largest JPEG** – at a fixed quality, more fine detail makes a bigger file, so the largest frame is the sharpest. Optional flash LED.
- **Uploading**: `POST /api/iot/captures/<capture_id>` with the JPEG bytes as the body (up to 1.5 MB on the device; the backend accepts 2 MB) and a 20-second timeout.

7.7 Wi-Fi – `connectWifi()`

- At start-up it tries for 20 seconds (and keeps reading the scale while waiting). If that fails it opens **PostAssist-Scale**.

- In the loop it retries every 15 seconds, and closes the set-up network once it is connected.
- Wi-Fi power saving is switched off for steadier timing. It can only use **2.4 GHz** networks.

PART 8

The device's own web page and serial commands

Tools for installing and checking the device without the main system.

8.1 The web page – `http://<device-IP>/`

Shows the live weight, buttons for Tare, Calibrate, Reset calibration and force tare, a test photo, the **focus helper** and the **Network** settings box. Behind it are these addresses:

Address	What it returns or does
`GET /`	The page itself.
`GET /status`	Everything in JSON: grams, spread, raw, stable, loaded, placement, sensor_ok, spikes ignored, calibration ID, factor, offset, camera, Wi-Fi, backend address, `backend_ok`, firmware. **Check this first** when something is wrong.
`GET /weight`	Grams and stable (same answer as the old pull sketch, used by `device_bridge.py --check`).
`POST /tare` (`?force=1`)	Tare (force = even when not stable).
`POST /calibrate?g=100`	Calibrate with that many grams.
`POST /calibrate/reset`	Back to the default factor, calibration cleared.
`GET /capture` or `/capture.jpg`	A test photo.
`GET /focus`	The size of one quick frame – the "detail score" of the focus helper. Turn the lens until it peaks.
`POST /settings`	Saves Wi-Fi name, password and backend address, then restarts.

8.2 Serial commands (board on the MB, Serial Monitor at 115200)

Type	Effect
`i`	Information: grams, spread, stable, loaded, placement, raw, offset, factor, calibration, spikes, IP, backend.
`t` / `tf`	Tare / force tare.
`c 100`	Calibrate with a 100 g mass.
`r`	Reset calibration.

PART 9

What the backend does with it

The server side of the IoT part (FastAPI, `backend/app/routers/iot.py`, `devices.py`, `services/weighing.py`).

9.1 The three doors the device uses

Endpoint	What the backend does
`POST /api/iot/readings`	1. Checks the device ID and key: it compares the **SHA-256 fingerprint** of the key with the stored one. Wrong → **401** and a **security** entry in the audit log. 2. Saves the reading as the device's `latest_reading`, with the time it arrived, and updates last seen, IP, firmware, Wi-Fi signal and camera status. 3. Keeps a history copy in `device_readings` when the platform is loaded, when something changed, or every 30 s while idle. History is **deleted automatically after 3 days**. 4. If the calibration ID is new, records the device as **calibrated** and clears the old ± uncertainty; if the device reports no calibration, marks it **uncalibrated**. 5. Expires commands that waited more than **60 s**, and returns up to **3** waiting commands (marking them "sent").
`POST /api/iot/captures/<id>`	Accepts the JPEG only for a capture that was requested for this device and is still open; max 2 MB. Saves it, runs the **image-quality check** (resolution, sharpness, brightness, contrast, glare) and attaches it to the draft if the draft is still open.
`POST /api/iot/commands/<id>/result`	Saves done/failed. A finished calibration is stored with its ID, factor, reference mass and who asked for it. A failed capture marks the capture failed. Every result goes to the audit log.

9.2 The doors the web app uses

Endpoint	Used for
`GET /api/devices`, `/{id}/live`	Scale & camera page: online/offline (offline after **10 s** without a reading), latest reading, last 5 commands. Refreshed every 0.8 s.
`POST /api/devices/{id}/commands`	Queue tare, calibrate (10–1000 g) or capture.
`/{id}/trials`, `/trials.csv`, `/uncertainty`	Weighing trials and the expanded uncertainty U (Part 10).
`POST /api/devices`, `/{id}/rotate-key`	Administrator: register a device or make a new key – the key is shown **once**.

Endpoint	Used for
`GET …/drafts/{id}/weight/live`	The counter's live weight panel (every 0.7 s) with "ready" or the first problem.
`POST …/drafts/{id}/weight/pair`	**Use this weight** – checks the rules below.
`POST …/drafts/{id}/capture`	**Capture from scale camera** – queues a capture command; expires after 25 s if no photo arrives.

9.3 The rules for "Use this weight"

Problem code	When it blocks the weight
`NO_READING`	The device never reported.
`STALE_READING`	The last reading is older than **3 s**.
`SENSOR_FAULT`	The HX711 is not answering.
`NEEDS_TARE`	Only with an old bridged board after a restart (firmware 1.1 keeps its zero in flash).
`UNCALIBRATED`	No calibration ID.
`PLATFORM_EMPTY` · `UNSTABLE` · `NEGATIVE`	Not loaded · spread over 1 g · weight below zero.
`PLACEMENT_ALREADY_USED`	The **placement key** `device : boot ID : placement` is already on another draft or item – the database allows each key **once** (test T03).

The paired weight keeps its device, calibration ID, spread, placement key and an **uncertainty**: the measured expanded uncertainty U (or 2 g by default), but never less than half the spread. The tariff engine uses it: if *weight* ± *uncertainty* crosses a price step (e.g. 29.5 g ± 1 g across 30 g), the answer is **"Reweigh required"**.

PART 10

Calibration and accuracy testing

How you make the scale correct, and how you prove how correct it is (objective SO2).

10.1 Calibrating

1. Let the scale warm up for about 5 minutes on a firm, level table with no fan.
2. Platform **empty**, wait for **stable**, press **Tare** (Scale & camera page, device page or `t`).
3. Put a reference mass (100 g or 200 g) in the **centre**, wait for stable, enter its mass and press **Calibrate**. A new calibration ID appears.
4. Check with other masses (20 g, 50 g, 500 g) – they should read within about ±1 g.
5. Re-tare at the start of each counter session (temperature drift).

10.2 Weighing trials (the 10 × 10 protocol)

1. Reference masses: **5, 10, 20, 30, 50, 100, 200, 300, 400, 500 g**; each weighed **10 times**, lifting it off between repeats.
2. On the **Scale & camera** page: enter the reference mass and condition (centre, off-centre, re-placed, after restart, near a band boundary), wait for stable, press **Record current stable reading**. The page shows the count and the **MAE**.
3. **Download trials CSV** and run `ml/evaluation/weighing_analysis.py` on it.
4. The report gives, per mass, the **bias** (average error) and **repeatability** (standard deviation), the overall MAE, a straight-line fit, and the **expanded uncertainty U**.
5. A postmaster enters U, with the evidence, on the Scale & camera page.

HOW U IS CALCULATED (K = 2)

$U = 2 \times \sqrt{(\text{largest } |\text{bias}|)^2 + (\text{largest standard deviation})^2}$

Example: largest bias 0.4 g, largest standard deviation 0.3 g → $U = 2 \times \sqrt{0.16 + 0.09} = 2 \times 0.5 =$ **1.0 g**. The target of the proposal is **MAE ≤ 1 g** over 5–500 g.

BE HONEST ABOUT THE REFERENCE

The accuracy you can claim is never better than the reference masses you used – report their class or tolerance. 62 trials were recorded on 11 October with the earlier board sketch; because firmware 1.1 has a new calibration, repeat the full 10 × 10 trials before quoting a final MAE or U.

PART 11

Using it at the counter

The officer's view.

1. **Counter intake → Register a new letter.**
2. Put the envelope **address side up** on the platform.
3. **Capture from scale camera** → the photo appears → drag a box around the address → **Read address**.
4. Check the fields → **I have checked the envelope – confirm details**.
5. In **Weight**, wait for Stable – ready → **Use this weight**.
6. **Calculate charge** → payment → **accept & print label** → scan the label to verify.
7. **Lift the letter off** before the next one – that starts a new placement.

If the scale is offline, broken, or the letter is over 500 g, the officer uses **Enter the weight by hand (approved instrument, audited)** (grams, instrument, reason). It is recorded in the audit log as a warning, so the counter

never stops.

PART 12

Security and reliability

Why the IoT data can be trusted.

What	How it is done
Only the real device can send weights	Every request carries the device ID and secret key. The server stores only the key's SHA-256 fingerprint and compares in constant time. Wrong key → 401 + security audit. An administrator can make a new key at any time.
A previous letter's weight cannot be reused	Placement numbers + a unique placement key in the database (test T03).
Old or shaky readings cannot be used	Fresh ≤ 3 s, stable ≤ 1 g spread, sensor OK, calibrated.
Every weight can be traced	Device, calibration ID, placement, spread and uncertainty are stored with the paired weight; commands and calibrations are audited.
Commands cannot pile up	Commands expire after 60 s; a capture expires after 25 s.
No open port needed on the device for the system	The device always starts the conversation, so it works behind a phone hotspot or router.
Encryption possible	With an `https://` backend and `ROOT_CA_PEM`, readings and photos are encrypted and the server is verified. The lab uses plain HTTP on a private hotspot.
Secrets kept out of git	`config.h` is git-ignored. (See Part 14: the template file currently holds a real Wi-Fi password.)
Weights survive power cuts	Calibration, zero point and placement number are in flash.

PART 13

Troubleshooting

Symptom → cause → fix. Start with the device's `/status` page.

Symptom	Likely cause	Fix
Upload: "Failed to connect to ESP32"	Board not in download mode; charge-only cable	Hold IO0, tap RST, release IO0, upload; use a data cable.
"ArduinoJson.h: No such file"	Library missing, or sketchbook on a path with special characters	Install ArduinoJson 7; move the sketchbook to a plain path.
Endless resets, "Brownout detector"	Weak 5 V supply	5 V / 2 A charger, short thick cable, 1000 µF capacitor.
`[cam] init failed`	Camera ribbon loose	Re-seat the ribbon, power-cycle.
Board does not boot after wiring	Something on IO12 or IO0	Use IO13/IO14 only.
`sensor_ok: false`	HX711 not answering	Check DT/SCK through LV1/LV2, LV = 3.3 V, HV = 5 V, HX711 power.
Weight goes down when pressed	A+ / A− swapped	Swap white and green, recalibrate.
Weight jumps between fixed values; many `spikes_ignored`	Bit errors through the shifter or long wires	Shorter DT/SCK wires; raise `HX711_HALF_PERIOD_US` to 8; or Option B.
Never stable, huge numbers (e.g. −5,100 g)	Bad stored factor (seen on 18 Oct: −12.7 counts/g)	Firmware 1.1.1: **Reset calibration** (`r`), force tare (`tf`), then calibrate again.
Wi-Fi never connects	5 GHz network or wrong password	Use 2.4 GHz / the hotspot; fix it on the device page or via PostAssist-Scale.
Serial: `readings -> -1`	Wrong backend address, backend on 127.0.0.1, firewall	Laptop's hotspot IP, `--host 0.0.0.0` , allow port 8000 on private networks.
Serial: `readings -> 401`	Device key mismatch	Copy the key from `backend/.env` into `config.h` .
Photo blurry	Lens focused far away	Focus helper: turn the lens until the score peaks; more light.
Capture "expired"	Camera off or slow upload	Check camera ok; use UXGA frame size; stronger Wi-Fi.
Readings look doubled or jump between two sources	The **bridge** is running as well as firmware 1.1	Empty `BRIDGE_DEVICE_URL` ; don't start the bridge.
`PLACEMENT_ALREADY_USED`	The last letter is still on the platform	Lift it off, wait for 0 g, place the new letter.

PART 14

Your device right now – to-do list

Based on the last checks (18–19 October 2026). Do these before the demo.

#	What	Why
1	**Flash firmware 1.1.1** (the version in the repository)	The board was last seen running 1.1.0. 1.1.1 adds Reset calibration and force tare, which fix the "never stable" lock seen on 18 October. It was compiled but not flashed (no USB connection at the time).
2	**Empty** `BRIDGE_DEVICE_URL` in `backend/.env` and do not run the bridge	It is currently set to `http://172.20.10.2`, so `start_all` also starts the bridge. With firmware 1.1 the board already reports by itself, so two sources send readings for `SCALECAM-01` at once.
3	**Reset, tare and calibrate**	Fresh calibration with a known mass; check 20 / 50 / 200 g.
4	**Focus the lens** for 30 cm	Sharp photos are what make the address readable.
5	**Repeat the 10 × 10 trials**, run the analysis, enter U	Evidence for SO2 with the current calibration.
6	**Remove the real Wi-Fi password** from `config.example.h` and change the hotspot password	The template is tracked by git and has been pushed to GitHub with your real hotspot name and password. Put placeholders there; your own `config.h` keeps the real values.

PART 15

Limitations

Say these openly – they show you understand the device.

- **Not legal for trade.** A DIY load cell is a research demonstrator (OIML R 76-1). Official charging must use an approved scale – supported by the audited manual weight.
- **Range.** 1 kg cell; the system is tested for 5–500 g. Heavier letters use the manual weight.
- **Environment.** Temperature drift, vibration, air flow and an off-centre letter affect the reading; re-tare each session.
- **The device page has no password.** Anyone on the same Wi-Fi could tare it or change its settings. Fine on a private hotspot, not for a real post office (would need a PIN or disabling the page).
- **Plain HTTP in the lab.** HTTPS is supported but not used on the hotspot.
- **Camera.** A fixed-focus 3 MP module at 30 cm; small or faint handwriting can still be hard to read.
- **One device per office** in the pilot; the backend supports more.

PART 16

Questions the panel may ask

Short, honest answers.

Q Why did you choose the ESP32-CAM?

It has Wi-Fi, a camera and PSRAM on one cheap board, so one device can weigh and photograph the same envelope at the same place.

Q Why do you need the HX711?

The load cell gives only a few millivolts. The HX711 amplifies it (gain 128) and converts it with 24-bit resolution, about 10 times a second – the ESP32's own converter is far too coarse and noisy for that.

Q Why the level shifter?

The HX711 runs at 5 V for a stronger load-cell signal, but ESP32 pins are 3.3 V and are damaged by 5 V. The shifter translates the data and clock lines.

Q How do you get grams from the sensor?

grams = (raw count − tare offset) ÷ calibration factor. The offset is the empty reading; the factor is counts per gram, measured with a known mass. Both are saved in flash.

Q How do you know the weight is reliable?

I keep a one-second window of 10 readings, use a trimmed mean, and call it stable only if the spread is 1 g or less. Single spikes are ignored unless the next reading confirms them.

Q How do you stop the previous letter's weight being used again?

The firmware numbers every placement – it only increases after the platform was empty. The database allows each device-boot-placement key on one letter only.

Q How does the device talk to the server?

It sends an HTTP POST with JSON every 500 ms, with its device ID and key in the headers. The server's reply carries any commands, so the server never has to connect to the device.

Q Why push instead of the server polling the device?

The device doesn't need an open port or a fixed IP, so it works behind a hotspot or router, and the server knows exactly when it last heard from it.

Q How is the device authenticated?

A per-device secret key. The server stores only its SHA-256 fingerprint; a wrong key is rejected and logged as a security event, and the key can be rotated.

Q How do you get sharp photos?

The lens is focused for 30 cm with a focus helper, the camera uses sharpening with noise reduction off, and it takes a burst of three and sends the largest JPEG, which is the most detailed.

Q Why didn't you use an HX711 library?

Through the level shifter the library's fast clock read wrong bits, so I read the chip myself with slower 4 µs pulses inside a critical section.

Q What happens if Wi-Fi drops?

The device reconnects every 15 s. Meanwhile readings become older than 3 s and cannot be paired, the Scale & camera page shows offline after 10 s, and the officer can use a manual weight.

Q How accurate is it?

I measure it with 10 reference masses × 10 repeats. The target is MAE ≤ 1 g; the analysis also gives an expanded uncertainty U, which the tariff engine uses to demand a reweigh near a price boundary.

Q What if a power cut happens?

Calibration, zero point and placement number are in flash, so the scale continues correctly; the new boot ID keeps placement keys unique.

Q What are its limitations?

Not legal for trade, 1 kg range, sensitive to drift and vibration, and the device page has no password – acceptable for a pilot on a private network.

PART 17

IoT words in plain language

Word	Meaning
Microcontroller	A small computer on one chip that runs one program (here the ESP32).
Firmware	The program inside the device.
Flash (uploading)	Writing the firmware into the device.
GPIO	A general-purpose pin the program can switch or read (e.g. GPIO 13).
Strain gauge · load cell	A sensor whose resistance changes when stretched · a metal bar carrying strain gauges.
ADC	Analogue-to-digital converter: turns a voltage into a number.
Raw count	The number from the HX711 before it is turned into grams.
Tare · offset	Setting zero · the raw count of the empty platform.
Calibration factor	Raw counts per gram.
Bit-banging	Reading or sending data one bit at a time by switching pins in the program.
Level shifter	Translates 5 V signals to 3.3 V and back.
PSRAM	Extra memory on the ESP32-CAM, needed for large photos.
NVS / Preferences	The ESP32's small storage area in flash that keeps values after power-off.
Serial monitor · baud	The laptop window showing the device's messages · the speed (115200).
HTTP POST · JSON	Sending data to a server · the text format of that data.

Word	Meaning
Header	Extra labelled information sent with a request (here the device ID and key).
RSSI	Wi-Fi signal strength in dBm (−50 is strong, −80 is weak).
Access point (AP)	A Wi-Fi network the device creates itself (PostAssist-Scale).
MAE · bias · repeatability	Average size of the errors · average error (direction) · how much repeats differ.
Expanded uncertainty (U, k = 2)	A "±" range that the true weight lies in with about 95 % confidence.
Brownout	A reset caused by the supply voltage dropping for a moment.